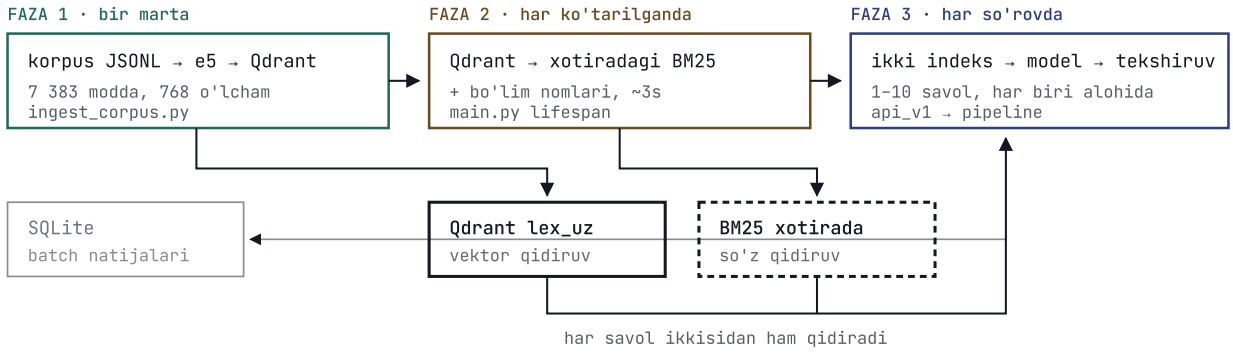
LEX.UZ RAG API

Algoritm qadamba-qadam

Har bir qadamda: nima bo‘ladi, qaysi fayl bajaradi, qanday texnologiya ishlatilgan, va nega aynan shu.

Loyiha uch fazada ishlaydi. Birinchisi **bir marta** bajariladi (indeks qurish), ikkinchisi **server har ko‘tarilganda**, uchinchisi **har so‘rovda**. Quyida uchalasi tartib bilan.



Ikki indeks ataylab: biri ma‘no bo‘yicha, ikkinchisi so‘z bo‘yicha qidiradi. SQLite esa natijani saqlab, takroriy so‘rovni qayta hisoblashdan saqlaydi.

FAZA 1 · BIR MARTA

Indeks qurish

Buyruq: `python ingest/ingest_corpus.py` . CPU da bir soatdan ko‘proq: o‘lchangani **~690 ms har modda**, ya‘ni 7 383 modda uchun **~85 daqiqa** (o‘lchov paytida mashina server bilan bo‘lishgan, bo‘sh mashinada tezroq). GPU bo‘lsa bir necha daqiqa bo‘lardi — bu loyihada GPU foyda beradigan *yagona* joy, va u bir martalik.

1.1 **Korpus fayllarini qatorma-qator o‘qish**

`ingest_corpus.py · iter_records()` `json (stdlib)`

Nega JSONL: har qator — mustaqil modda. Fayl 10 MB bo‘lsa ham, hammasini xotiraga olish kerak emas, va yarmida uzilsa qolgani buzilmaydi.

1.2 **64 taligdan to‘plab, modda sarlavhasini matn boshiga qo‘shish**

`ingest_corpus.py · _flush()`

Nega sarlavha matn ichida: «217-modda» raqami vektorga ham tushsin — aks holda modda raqami bo‘yicha qidiruv ishlamaydi. **Nega 64 ta:** model bir vaqtda paketni tezroq hisoblaydi, lekin xotira o‘smasin.

1.3 **Har bo‘lakni 768 raqamli vektorga aylantirish**

`embedding.py · embed_passages()` `sentence-transformers` `intfloat/multilingual-e5-base`

Nega e5: avval MiniLM sinalgan — u har nomzodga 0.69–0.71 ball berib, hech narsani farqlamagan va to‘g‘ri moddani butunlay o‘tkazib yuborgan. e5 aynan qidiruv uchun o‘qitilgan va o‘zbek/rus tilini qoplaydi. **Nega mahalliy:** API kaliti kerak emas, pul ketmaydi, CPU da ishlaydi. **Nega «passage: » prefiksi:** e5 shuni talab qiladi — indekslanadigan matn uchun `passage:` , qidiruv uchun `query:` . Aralashtirilsa aniqlik tushadi.

1.4 **Vektor + payload ni bazaga yozish**

`ingest.py · qdrant_client()` `qdrant-client` `Qdrant (Docker)`

Nega Qdrant, Chroma emas: ikki narsa kerak edi — payload bo‘yicha filtr (`code` , `source`) va butun kolleksiyani `scroll` bilan qaytib o‘qish. Qdrant ikkisini ham yaxshi bajaradi, va Docker’da bitta qatorda ko‘tariladi. **Payload nima uchun:** `code` , `article_id` , `section` , `lex_uz_doc` — sitatadagi modda, qism va havola shulardan quriladi.

1.5 **Qayerga yetganini faylga yozib borish**

`ingest_corpus.py · progress_file`

Nega: 7 383 moddani vektorga aylantirish uzoq. Uzilsa, qayta yurgizganda boshidan emas, qoldirgan joydan davom etadi.

FAZA 2 · HAR KO‘TARILGANDA

Server ishga tushishi

Buyruq: `uvicorn backend.main:app --port 8000` . ~30 sekund.

2.1 **SQLite jadvalini yaratish va eskirgan yozuvlarni o‘chirish**

`store.py · init(), purge_expired()` `sqlite3 (stdlib)`

Nega SQLite: kerak bo‘lgani — bitta tranzaksiyali jadval: `batch_id` band qilish, takrorni aniqlash, async holatni saqlash. Buning uchun alohida baza servisi ortiqcha. **Nega WAL rejimi:** fonda ishlayotgan batch yozayotganda `GET` bir vaqtda o‘qiy olsin. **Nega 30 kun:** spec talabi.

2.2 **Qdrant dan barcha 7 383 modda payload‘ini o‘qish**

`retrieval.py · load_articles()` `Qdrant scroll`

Nega hammasini: BM25 indeksini qurish uchun barcha matn kerak. **Nega filtr bilan:** faqat `source = uzbek_legal_corpus` olinadi — eski skraper bo‘laklarida modda strukturasi yo‘q va ulardan haqiqiy sitata qurilmaydi.

2.3 **Tiklangan bo‘lim nomlarini moddalarga qo‘shish**

`retrieval.py · _full_section()` `data/section_titles.json`

Nega kerak: korpus bo‘limni `"X BO‘LIM."` deb saqlagan, nomini tashlab ketgan. Soliq kodeksida 13 modda «Soliq stavkalari» deb nomlangan va hammasi stavkani nomsiz aytadi — qaysi soliq ekani faqat `QO‘SHILGAN QIYMAT SOLIG‘I` sarlavhasidan bilinadi. Shu nom bo‘lmaganda QQS savoli noto‘g‘ri «javob yo‘q» bergan.

2.4 **BM25 teskari indeksini xotirada qurish**

`retrieval.py · LexicalIndex.__init__()` `o‘z implementatsiyasi`

Nega xotirada: korpus bir necha o‘n MB. Alohida qidiruv servisi (Elasticsearch kabi) hech nima qo‘shmaydi, lekin o‘rnatish va boshqarishni qo‘shadi. 3 sekundda quriladi. **Nega o‘z kodi:** kerak bo‘lgani — tf, idf, hujjat uzunligi va teskari indeks; bu ~40 qator. Kutubxona tokenizatsiyani o‘zi boshqargan bo‘lardi, bizga esa o‘zbek tiliga xos tokenizatsiya kerak (stopword, sinonim, transliteratsiya). **Sarlavha ikki baravar:** modda sarlavhasi uning eng farqlovchi qismi va qisqa.

2.5 **e5 modelini oldindan yuklash**

`main.py · lifespan → embed_query("warmup")`

Nega: model birinchi ishlatilganda yuklanadi. Bu qilinmaganda birinchi savol **8.4 sekund** ketgan, keyingilari 0.24 sekund. O‘lchangan va tuzatilgan.

FAZA 3 · HAR SO‘ROVDA

Darvozalar

Bu qadamlar qidiruv boshlanmasdan, **batch darajasida** bajariladi.

3.1 **So‘rov shaklini tekshirish**

`schemas.py · BatchRequest` `pydantic` `FastAPI`

Nega FastAPI: so‘rov sxemasi Python tiplari bilan yoziladi va validatsiya avtomat bo‘ladi, plus `/docs` da OpenAPI o‘zi chiqadi. **Nima tekshiriladi:** 1–10 savol, takroriy `id` , bo‘sh savol, 2000+ belgi → `400` . **Nega bu yerda:** hech qanday vektor hisoblanmasdan, hech qanday pul ketmasdan rad etiladi.

3.2 **Bearer kalitni tekshirish**

`api_v1.py · require_api_key()`

Nega 401 va 403 ajratilgan: header yo‘q yoki buzuq → `401` (autentifikatsiya qilinmagan); header to‘g‘ri, lekin kalit tanilmadi → `403` . Testerga nimani tuzatish kerakligini aytadi.

3.3 **Sekundda bitta batch chegarasi**

`store.py · RateLimiter.check()`

Nega: spec «kamida 1 batch/sekund» talab qiladi. Oshib ketsa `429` va `Retry-After` — tester qancha kutishni biladi, taxmin qilmaydi.

3.4 **Savollardan «barmoq izi» hisoblash**

`api_v1.py · _request_hash()` `hashlib sha256`

Nega: bir xil `batch_id` ikki xil sababdan kelishi mumkin — tester qayta urinyapti (bir xil savollar), yoki xato qilib `batch_id` ni qayta ishlatyapti (boshqa savollar). Hash ikkisini farqlaydi. Sozlamalar ham hashga kiradi: bir xil savollar boshqa `top_k` bilan — boshqa run.

3.5 **`batch_id` ni band qilish**

`store.py · reserve()` `SQLite BEGIN IMMEDIATE`

Uch natija: yangi → hisoblanadi; mavjud va hash bir xil → **saqlangan natija qaytadi**, qayta hisoblanmaydi; mavjud, hash boshqa → `409` . **Nega tranzaksiya:** ikki bir vaqtda kelgan so‘rov ikkalasi ham hisoblashni boshlab yubormasligi uchun.

3.6 **Ishni fon oqimiga berish va 120 sekund kutish**

`api_v1.py · ThreadPoolExecutor`

Nega oqim: ish 120 sekundda tugasa — `200` qaytadi. Tugmasa, **ish davom etadi** va `202` qaytadi; tester keyin `GET` bilan oladi. Spec aynan shuni talab qiladi. Ya‘ni sekin batch yo‘qolmaydi.

FAZA 3 · HAR SAVOL UCHUN

Qidiruv va javob

Endi batch ichidagi har bir savol alohida shu yo‘ldan o‘tadi.

4.1 **Savol qaysi tilda yozilganini aniqlash**

`lang.py · detect()`

Qanday: yozuv lotinmi kirillmi — bu to‘g‘ridan ko‘rinadi. Kirill ichida o‘zbek va rusni faqat bir tilda bor harflar ajratadi: `ў қ ғ ҳ` → o‘zbek, `ы э щ ъ ё` → rus. **Nega model emas:** test savollari qisqa, bu usul yetarli va hech nima yuklab olmaydi. **Yana shu yerda:** apostroflar birlashtiriladi — klaviatura `‘ ’ ʻ ʼ ` ´ '` dan bittasini beradi, korpus esa faqat `‘` ishlatadi.

4.2 **Savolni so‘zlarga ajratish va tozalash**

`retrieval.py · tokenize()`

Stopword: «kodeksiga», «ko‘ra», «qanday», «qancha» deyarli har savolda bor — ular qidiruvga signal bermaydi. Kodeks nomlarining o‘zagi ham («jinoyat») — u shu kodeksning deyarli har moddasida uchraydi va farqlamaydi. **Transliteratsiya:** korpus lotin yozuvida, shuning uchun `ўғрилик` javob beradigan modda bilan (`o‘g‘rilik`) bitta ham token baham ko‘rmaydi. Kirill token lotinga o‘giriladi va ikkisi ham saqlanadi. **Sinonim:** `qqs → qo‘shilgan qiymat` , chunki statut qisqartmani hech qachon yozmaydi.

4.3 **Savol qaysi kodeksni atayotganini aniqlash**

`retrieval.py · detect_codes()` `re (regex)`

Nega: «Mehnat kodeksiga ko‘ra» deyilgan bo‘lsa, qidiruvni shu kodeksga yo‘naltirish kerak — aks holda umumiy moddalar (jinoyat tushunchasi, jazo turlari, protsessual qoliplar) so‘z ustma-ustligi bilan aniq moddani bosib ketadi. **Uch tilda:** «mehnat kodeks», «меҳнат кодекс», «трудов* кодекс» — hammasi bitta qoidada. **Tartib muhim:** «jinoyat-protsessual» «jinoyat» dan oldin tekshiriladi, aks holda har protsessual savol jinoyat kodeksiga tushadi.

4.4 **Savolni 768 raqamli vektorga aylantirish**

`embedding.py · embed_query()` `e5, "query: " prefiksi`

Nega boshqa prefiks: e5 modeli savol va indekslangan matnni boshqacha kodlaydi. 1.3-qadamda `passage:` ishlatilgan, bu yerda `query:` . Shuning uchun ikkisi bitta faylda turadi — adashtirib bo‘lmaydi.

4.5 **Uch manbadan nomzod to‘plash**

`retrieval.py · _dense_search(), index.search()`

Dense (40 ta): ma‘no bo‘yicha — «ishdan bo‘shatishsa» → «mehnat shartnomasini bekor qilish». **Dense kodeks ichidan (40 ta):** global top-40 o‘tkazib yuborgan in-kodeks moddalarni olib keladi. **BM25 (40 ta):** kam uchraydigan aniq atama — «o‘g‘irlik» → sarlavhasi «O‘g‘rilik». **Nega ikkisi ham:** dense parafrazani topadi lekin aniq atamani o‘tkazib yuboradi; BM25 teskarisi. Bittasi yetmaydi.

4.6 **Ballarni birlashtirish va top-5 ni tanlash**

`retrieval.py · retrieve() fusion bloki`

Bu loyihadagi eng muhim qaror — pastda alohida tushuntirilgan.

4.7 **Modelga yuborish uchun kontekst qurish**

`generate.py · build_prompt()`

Har kontekst bloki: `document` (kodeks nomi), `location` (bo‘lim + bob), `article` (modda va sarlavhasi), `text` (modda matni). **Nega `location` :** moddaning kodeksdagi o‘rni uning ma‘nosining bir qismi. Bu qator bo‘lmaganda «soliq stavkasi 12 foiz» qaysi soliqqa tegishli ekani bilinmagan.

4.8 **Modeldan JSON so‘rash**

`generate.py · provayder yo’li` `anthropic SDK` `httpx` `JSON schema`

Nega matn emas, JSON: bizga mashina tekshiradigan narsa kerak — qaysi kontekstdan olgani va **harfma-harf ko‘chirgan parcha**. Erkin matn bu ikkisini ishonchli bermaydi. **Nega sxema majburlanadi:** provayder javob shaklini server tomonda tekshiradi, ya‘ni shakl buzilgan javob kelmaydi. **Nega ikki transport:** Claude uchun rasmiy SDK (uning o‘z xususiyatlari bor),

qolganlari uchun oddiy HTTP — chunki ular bir xil OpenAI protokolida gaplashadi va alohida kutubxona keraksiz.

4.9 **Vaqtinchalik xatoda qayta urinish**

`pipeline.py · _generate_with_retry()`

Nega: bepul tarifda `429` oddiy hol — Groq daqiqada 8 000 token beradi, bitta savol ~2 500 token oladi. Agar bu doimiy xato bo‘lib qolsa, bepul kalitdagi tizim pullik kalitdagi *aynan shu* tizimdan past baho olardi. 45 sekundlik budjet ichida, provayder `Retry-After` bersa shuni hurmat qilib qayta urinadi.

4.10 **Har bir sitatani tekshirish — darvoza**

`pipeline.py · verify_citations()`

Pastda alohida. Bu javobni ishonchli qiladigan qism.

4.11 **Belgilarni qayta raqamlash va natijani yig‘ish**

`pipeline.py · renumber_markers()`

Nega: agar `[2]` sitatasi tekshiruvdan o‘tmay tashlangan bo‘lsa, javob matnidagi `[2]` belgisi hech narsaga ishora qilmaydi. Qolgan sitatalarga moslab qayta raqamlanadi, ya‘ni javobdagi har `[n]` haqiqiy `citations[].ref` ga mos keladi.

CHUQURROQ · 4.6-QADAM

Nega ball, o‘rin emas

Dastlab standart usul ishlatilgan edi: **RRF** (reciprocal rank fusion) — har qidiruvning *o‘rnini* qo‘shish. Bu korpusda noto‘g‘ri bo‘lib chiqdi.

Yuridik moddalar lug‘atni baham ko‘radi, shuning uchun e5 o‘xshashliklari zichlashadi: bitta savolning 40 nomzodi odatda **0.79–0.81** oralig‘ida. Ya‘ni dense *tartibi* deyarli tanga tashlash. BM25 farqi esa keng va ma‘noli: sarlavhaga aniq moslik **12.5**, keyingisi **7.9**.

RRF ball kattaligini tashlab yuboradi — aynan shu yerda farqlaydigan signalni. *O‘g‘irlik* savolining to‘g‘ri moddasi (Jinoyat kodeksi 169-modda) **5-o‘rinda ko‘milib qolgan**. Absolyut ball bilan 1-o‘ringa chiqdi.

```
fused = 1.0 · dense + 0.8 · leksik + 0.3 · kodeks_bonusi

dense   = clamp((cosine - 0.75) / 0.20, 0, 1)
leksik  = bm25 / (bm25 + 6)
kodeks  = 0.3 agar savol shu kodeksni atagan bo'lsa
```

HAD	SHAKLI	NEGA AYNAN SHUNDAY
`dense`	absolyut, nisbiy emas	Nomzodlar to‘dasiga nisbatan normallashtirilsa, bir-biriga o‘xshash o‘rtamiyona nomzodlar to‘dasi o‘zining eng yaxshisini 1.0 ga ko‘taradi — ya‘ni hech narsa topilmaganda ham «ishonchli» ball chiqadi

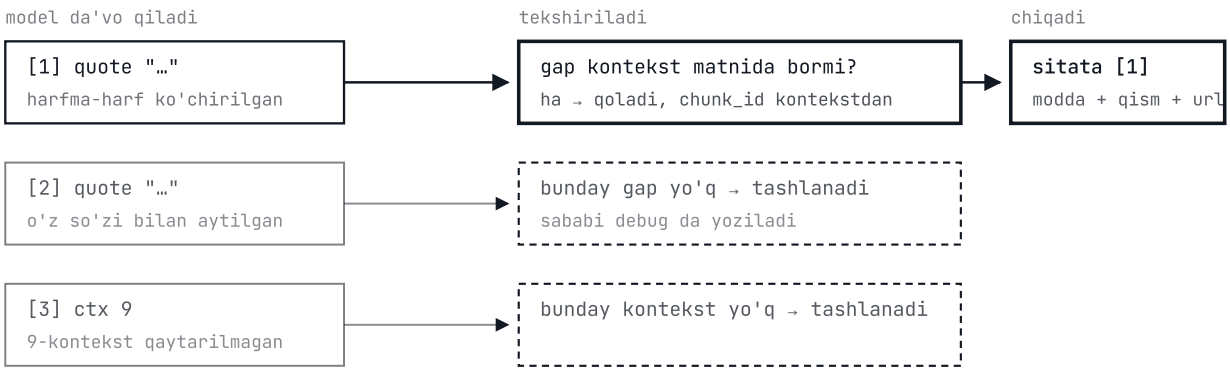
`leksik`	to‘yinuvchi	Kuchli aniq moslik (12.5) tasodifiy so‘z ustma-ustligidan (2.0) ancha ko‘p turishi kerak, lekin cheksiz o‘smasligi ham kerak
`kodeks`	tekis	Avval bu dense ballga *qo‘shimcha* hissa qo‘shardi, ya‘ni kodeks aniqlanganda dense vazni jimgina 1.0 dan 1.6 ga chiqardi — va cosine 0.845 bo‘lgan umumiy modda hal qiluvchi so‘z sarlavhasida turgan moddani bosib ketardi. Tekis bo‘lsa, bir kodeks ichida o‘zaro bekor bo‘ladi va qidiruvlarning o‘zi hal qiladi

Tartiblash `(-ball, chunk_id)` bo‘yicha — ya‘ni teng ballda ham natija har doim bir xil. Spec takrorlanuvchanlikni aynan shu yerda tekshiradi.

CHUQURROQ · 4.10-QADAM

Tekshiruv darvozasi

Model har bir da‘vo uchun «mana shu gapni shu kontekstdan ko‘chirdim» deydi. «Aytdi» va «qildi» bir narsa emas, shuning uchun tekshiriladi:



`chunk_id` kontekstdan o‘qiladi, modeldan hech qachon olinmaydi — shuning uchun tizim ko‘rmagan manbaga havolaning kelib chiqadigan joyi yo‘q.

> NEGA BU ARXITEKTURANING MARKAZI
>
> Yuridik savolda xato javob xavfli. Modelni «to‘g‘ri bo‘l» deb ishontirib bo‘lmaydi — lekin uning har bir da‘vosini manbaga qarshi tekshirib bo‘ladi. Shuning uchun to‘qilgan manba «tavsiya etilmaydi» emas, **strukturaviy imkonsiz**.
>
> 100 savolda o‘lchangan: faithfulness **1.000**, citation accuracy **0.966**, to‘qilgan manba **0**. Kuchsiz model bu yerda sitatani *yo‘qotadi*, o‘ylab topmaydi.

DAVOMI

Nega aynan shu texnologiya

Bu sahifa **qanday ishlashini** ko‘rsatadi. Har bir texnologiyaning **nima ekani** va **nega boshqasi emasligi** alohida sahifada — BM25 formulasi, Qdrant va uning alternativalari (Chroma, FAISS, pgvector, Pinecone), e5 va MiniLM, RRF nega yiqilgani, chunking usullari, SQLite va Postgres, nega LangChain emas:

Nega shu texnologiya →

Bu sahifa kodning haqiqiy ketma-ketligidan olingan: `api_v1.post_batch` → `pipeline.answer_question` → `retrieval.retrieve` → `generate.generate` → `pipeline.verify_citations`. Oxirgi tekshiruv: `verify.py --quick` → 6 o‘tdi, 0 yiqildi; 77 test o‘tadi.